

DATA STRUCTURES – MOCK TEST PAPER

Course Code: ETCCDS202

Course: Data Structures

Time: 3 Hours

Maximum Marks: 40

PART – A

Attempt ANY 4 Questions (2 Marks Each)

Q1. Differentiate between Array and Linked List on the basis of Memory allocation and Insertion operation.

Q2. Find the time complexity: $i = 1$ while $i \leq n$: $i = i * 2$

Q3. State whether the following statement is TRUE or FALSE: "Binary Search can be applied on unsorted arrays." Justify your answer.

Q4. What is Stack Underflow?

Q5. Write the inorder traversal rule of a Binary Tree.

Q6. Differentiate between BFS and DFS.

Q7. What is the worst-case complexity of Quick Sort?

PART – B

Attempt ANY 4 Questions (8 Marks Each)

Q1. Quick Sort Dry Run

Perform Quick Sort on the following array using the first element as pivot:

35, 10, 50, 25, 5, 40, 15

Show:

- Pivot selection
- Partitioning steps
- Recursive breakdown
- Final sorted array
- Time complexity

Q2. Queue Rearrangement

Using ONLY queue operations, transform:

1 2 3 4 5 6

into:

1 3 5 2 4 6

Show every intermediate queue state step-by-step.

Q3. Stack Operations

Consider the following operations:

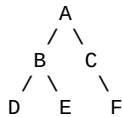
```
push(10)
push(20)
push(30)
pop()
push(40)
pop()
push(50)
```

Answer the following:

1. Show stack after every operation
2. What is the final stack?
3. What is the top element?
4. What happens if another pop operation is performed on an empty stack?

Q4. Binary Tree Traversal

Given the following Binary Tree:



Find:

1. Inorder Traversal
2. Preorder Traversal
3. Postorder Traversal

Show node-by-node tracing clearly.

Q5. BFS and DFS Traversal

Consider the graph:

```
A → B, C
B → D, E
C → F
E → G
```

Perform:

1. BFS Traversal
2. DFS Traversal

Starting from node A.

Also specify:

- Data structure used in BFS
- Data structure used in DFS

Q6. Construct Binary Tree

Construct the Binary Tree using:

Preorder:

A B D E C F

Inorder:

D B E A C F

Show:

- Root identification
- Left subtree

- Right subtree
- Recursive subdivision
- Final tree diagram

Q7. Complexity Analysis

Find the time complexity of the following code snippets:

(a)

```
for i in range(n):
    print(i)
```

(b)

```
i = 1
while i < n:
    i = i * 3
```

(c)

```
for i in range(n):
    for j in range(n):
        print(i, j)
```

Also classify each as:

- Best Case
- Average Case
- Worst Case

Q8. Searching Techniques

Differentiate between:

1. Linear Search
2. Binary Search

On the basis of:

- Working principle
- Time complexity
- Requirement of sorting
- Best use case

Also search element 25 in the following sorted array using Binary Search:

5, 10, 15, 20, 25, 30, 35

Show all intermediate steps.

Important Instructions:

- Write proper dry runs
- Show all intermediate steps
- Draw trees neatly
- Mention algorithms/pseudocode wherever required
- Do NOT jump directly to final answers